Real-Time PPE Detection

A computer vision system for construction site safety

Mini Project: Real-Time Computer Vision System

Name	Roll No.
Shantanu Vaidya	B2-20
Vineet Shah	B2-28
Mohammad Ejaz Mehendi	B2-25

Guide: Sweta Jain

In one paragraph. The system looks at images, videos and live camera feeds and detects workers, their safety gear (hardhat, vest, mask) and **missing** gear. When gear is missing it shows a red warning. We cleaned a public dataset, trained a YOLOv8n model (0.600 mAP@0.5, about 25 FPS on a laptop CPU), compared it with four state-of-the-art YOLO models and with published results, and built a web app that runs on laptops and phones.

How this document maps to the marking scheme

Criterion	Marks	Section	Evidence
Data cleaning and splitting	3	2	Quality checks, train/valid/test split, class balance chart
State-of-the-art model comparison and performance analytics	4	3	YOLOv8n results, per-class chart, comparison with 4 YOLO models and published work
UI deployment and cross-platform accessibility	4	4	Streamlit app with 4 input modes; laptop, phone and online access
Code quality, GitHub repo, report	4	5	One script per step, 10-page report

1. Problem and approach

Problem. On construction sites, supervisors check hardhats, vests and masks by walking around, which is slow and cannot be done all the time.

Goal. Detect people, their PPE and missing PPE in real time from images, videos and cameras, show a warning when PPE is missing, and measure accuracy with precision, recall and mAP.

Why YOLO. YOLO (You Only Look Once) looks at the whole image once and predicts all boxes and classes together, which makes it fast enough for live video.

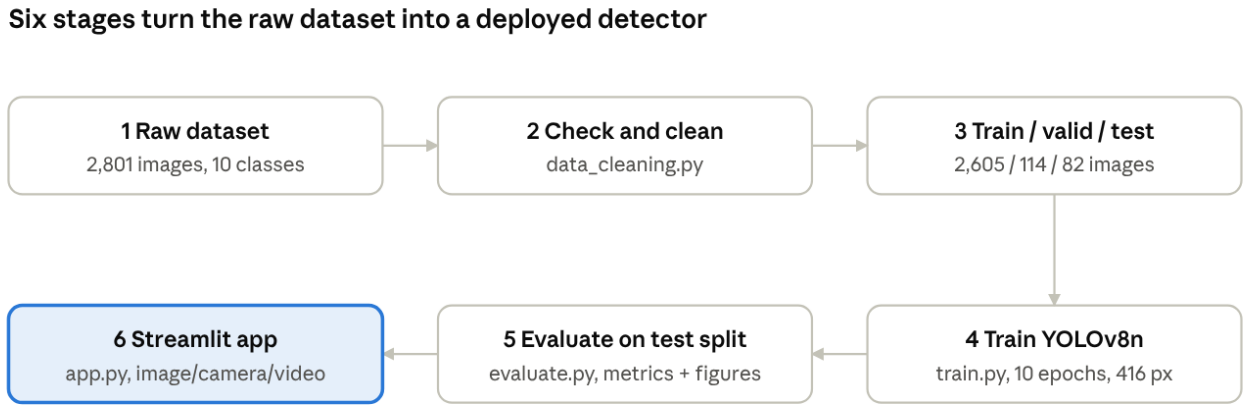


Figure 1. Project pipeline: six steps, one Python script each.

Dataset. Construction Site Safety (Roboflow, Kaggle copy): 2,801 images with 10 classes: Hardhat, Mask, NO-Hardhat, NO-Mask, NO-Safety Vest, Person, Safety Cone, Safety Vest, machinery, vehicle. The NO-* classes mean the item is missing and trigger the warning.

2. Data cleaning and splitting (3 marks)

2.1 Cleaning (data_cleaning.py)

Check	Result
Corrupt or unreadable images	0 of 2,801
Images without labels / labels without images	0
Invalid class numbers or box coordinates	0
Duplicate boxes in one file	2 (removed)
Images with no objects	24 (kept as background examples)

2.2 Splitting

The model learns from the training set, the validation set picks the best epoch, and the test set is used only for the final results.

Split	Images	Share	Boxes
Train	2,605	93%	36,893
Validation	114	4%	697
Test	82	3%	760

2.3 Class balance and preprocessing

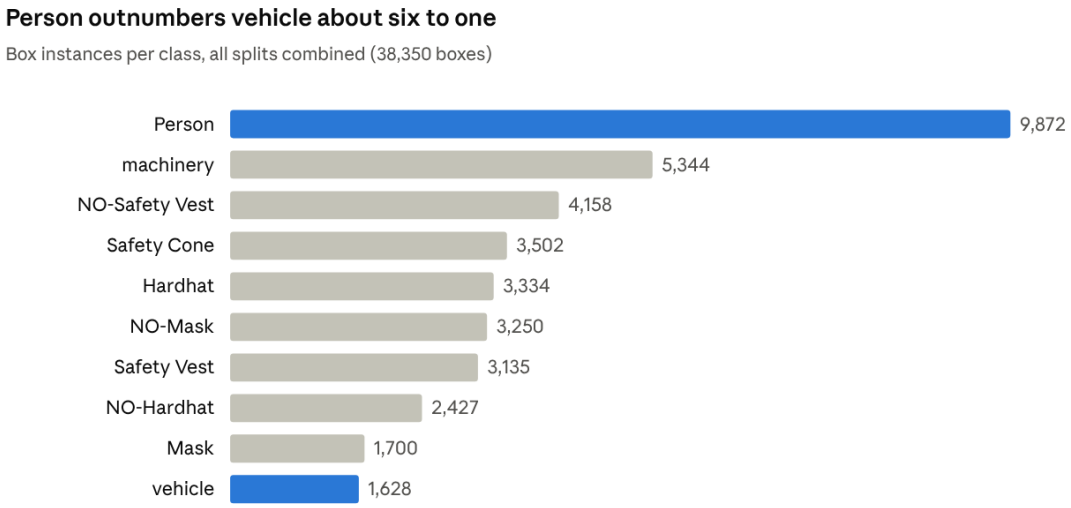


Figure 2. Boxes per class: Person (9,872) outnumbers vehicle (1,628) about six to one.

Rare classes have fewer examples, so they are harder to detect. **Preprocessing:** images are 640 x 640; during training YOLO scales pixels to 0-1 and applies augmentation (mosaic, flips, colour changes, scaling).



Figure 3. Sample images with labelled boxes (red = missing PPE class).

3. State-of-the-art model comparison (4 marks)

3.1 Our model

Setting	Value
Model	YOLOv8n (nano), COCO pre-trained, fine-tuned on PPE images
Epochs / image size / batch	10 / 416 x 416 px / 16
Hardware / training time	Laptop CPU (AMD Ryzen 7 7840HS), 0.77 hours

3.2 Metrics

Metric	Meaning
Precision = TP / (TP + FP)	Of the detections, how many are correct
Recall = TP / (TP + FN)	Of the real objects, how many were found
IoU	Overlap between predicted and true box; correct if at least 0.5
mAP@0.5	Average precision over the 10 classes at IoU 0.5 (main accuracy score)
mAP@0.5:0.95	Same, averaged over stricter overlap levels
FPS	Images processed per second (speed)

3.3 Results on the 82 test images

Precision	Recall	mAP@0.5	mAP@0.5:0.95	Speed (CPU)
0.786	0.530	0.600	0.299	~25 FPS

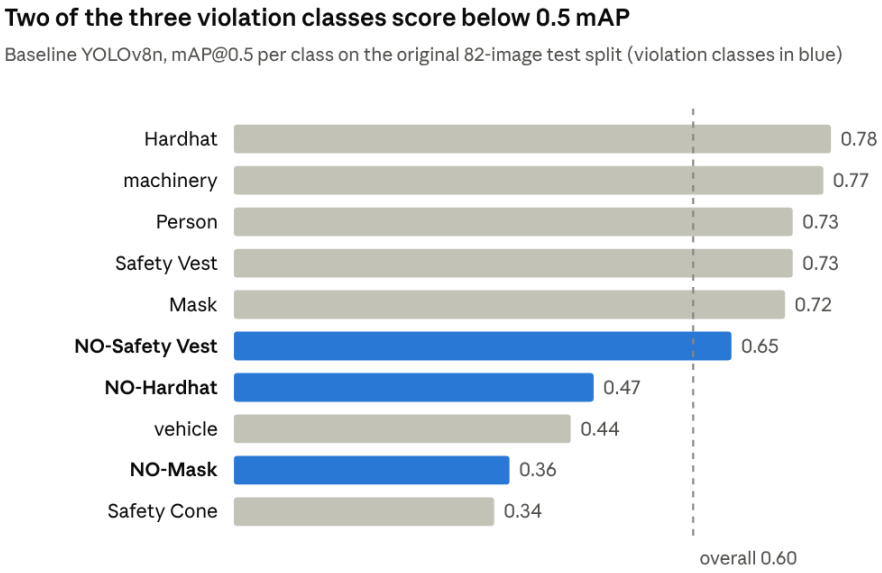


Figure 4. mAP@0.5 per class: two of the three missing-PPE classes score below 0.5.

Hardhat, machinery, Person and Safety Vest are detected well. NO-Mask, NO-Hardhat, Safety Cone and vehicle are harder: missing items are harder to see, and cones and vehicles are small or rare. Precision is higher than recall, so the model is usually right but misses some objects.

3.4 Comparison with 4 state-of-the-art models (COCO benchmark)

Model	Released	Params (M)	FLOPs (B)	COCO mAP@0.5:0.95
YOLOv5s (yolov5su)	2020	9.1	24.0	43.0
YOLOv8n (used here)	2023	3.2	8.7	37.3
YOLOv8s	2023	11.2	28.6	44.9
YOLO11n	2024	2.6	6.5	39.5

Larger models are more accurate but slower. YOLO11n is smaller and more accurate than YOLOv8n, so it is the next model to try. We chose YOLOv8n because it is fast. (Numbers from the Ultralytics documentation.)

3.5 Comparison with published PPE results

Study	Model	Dataset	Training	Precision	Recall	mAP@0.5
Ciftci et al. (2024)	YOLOv5n	Same dataset	20 epochs, 640 px, GPU	0.759	0.592	0.660
Ciftci et al. (2024)	YOLOv8n	Same dataset	20 epochs, 640 px, GPU	0.871	0.654	0.727
Feng et al. (2024)	YOLOv8s	103,500 images	200 iterations, GPU	0.850	0.605	0.840
This project	YOLOv8n	Same dataset	10 epochs, 416 px, CPU	0.786	0.530	0.600

Why 0.600 vs 0.727 for the same model: we trained for half the epochs (10 vs 20), with smaller images (416 vs 640 px) and on a CPU instead of a GPU.

4. UI deployment and cross-platform accessibility (4 marks)

The interface is a **Streamlit web app** (app.py). Users open it in a web browser, so it works on any device without installing anything.

Part	What it does
Image tab	Upload a photo; original and detected image side by side
Camera tab	Takes a photo with the device camera - works on phones

Part	What it does
Video tab	Detection on an uploaded video, frame by frame
Live webcam tab	Live detection from the laptop webcam with FPS
Warning banner	Red warning naming the missing PPE; green when everyone is safe
Sidebar	Model selection, confidence slider, class filter

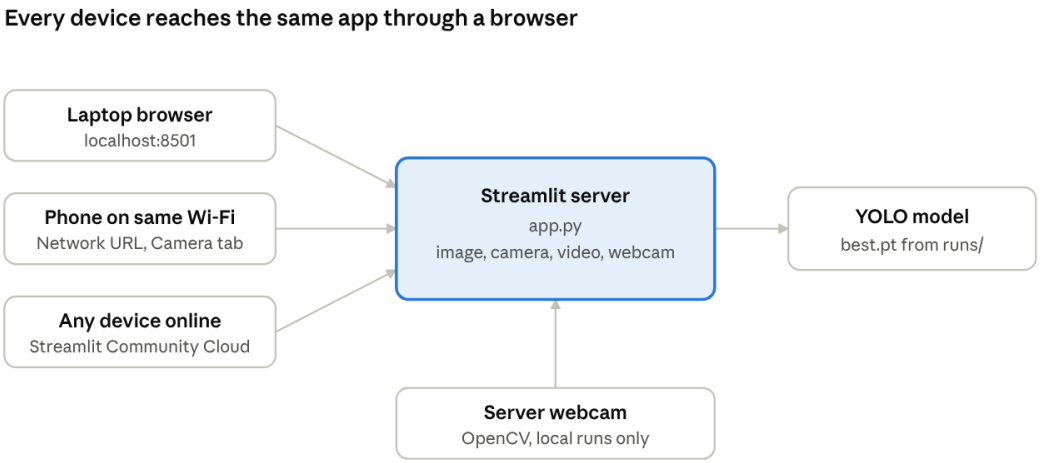


Figure 5. Ways to open the app; the live webcam works only when the app runs on the laptop.

Access	How
Same computer	python -m streamlit run app.py, then open localhost:8501
Phone / tablet on same Wi-Fi	python -m streamlit run app.py --server.address 0.0.0.0, open the printed Network URL
Anywhere online	Deploy the GitHub repo on Streamlit Community Cloud (requirements.txt and packages.txt are ready)

5. Code quality, GitHub repository and report (4 marks)

File	Purpose
data_cleaning.py	Checks images and labels, draws dataset figures
train.py	Trains the YOLO model
evaluate.py	Computes precision, recall, mAP and FPS on the test set
app.py	Streamlit web app (image, camera, video, live webcam)
requirements.txt, packages.txt	Packages needed to run locally and online

Report (10 pages): Introduction, Literature review (6 papers from 2024), Methodology, Data cleaning and preprocessing, Results and discussion, State-of-the-art comparison, UI development and deployment, Conclusion, References.

6. Literature review summary

Paper	Year	Focus	Key result
Wang, Yeh, Liao - YOLOv9	2024	General detection	PGI + GELAN: better parameter use
Zhao et al. - RT-DETR (CVPR)	2024	Transformer detector	53.1% AP at 108 FPS on T4, no NMS
Wang et al. - YOLOv10 (NeurIPS)	2024	NMS-free YOLO	1.8x faster than RT-DETR-R18 at similar AP
Khanam, Hussain - YOLO11	2024	Architecture review	C3k2, SPPF, C2PSA blocks
Ciftci, Turkdamar, Ozturk	2024	PPE, same dataset	YOLOv8n 0.727 mAP@0.5
Feng, Miao, Zheng	2024	PPE, 103,500 images	YOLOv8s 84.0% mAP@0.5, recall 60.5%

7. Limitations and future work

Limitation	Planned improvement
A normal cap is sometimes detected as a Hardhat	Add photos of normal caps to the training data
Short training (10 epochs on a CPU)	Train longer on a GPU; try YOLO11n and YOLOv8s
Small objects (cones, far-away workers) are missed	Use larger image sizes
Checks that PPE is present, not worn correctly	Check that the hardhat is on the head